

Searching for Solutions: From Competitive Code to Deep Research

How AlphaCode and AlphaCode 2 search the space of code solutions through massive sampling and learned scoring, and how Search-o1 applies the same search-and-refine idea to reasoning models doing deep research

Why Competitive Programming Is a Much Harder Problem

A typical coding agent today mostly does **autocomplete** — given a partial line, it completes it. That's a much easier problem than **competitive programming**, where you're given a full problem description (often long and requiring real reading comprehension) plus example inputs and outputs, and have to work out a complete, correct solution end to end — including figuring out *what approach to even take* before writing any code at all.

This is meaningfully different from something like a simple coding benchmark (e.g., "complete this function"), which mostly tests whether a model can translate a clear, narrow instruction into code. Competitive programming requires understanding a problem description, reasoning about the right algorithmic approach, and only then producing a full solution — genuinely closer to how a human competitive programmer works.

AlphaCode

Paper: arxiv.org/abs/2203.07814

Released by DeepMind in 2022 (an older paper by today's standards, but foundational for this kind of search-and-select approach), AlphaCode ranked in the **top 54.3%** of participants across 10 real competitions on Codeforces, a popular competitive programming platform — the first time an AI system demonstrated this kind of end-to-end, competition-level programming performance, rather than only solving narrow, well-specified tasks.

The High-Level Pipeline

1. **Pre-train and fine-tune a model** on code data (this was before today's large general-purpose LLMs, so AlphaCode trained its own model from scratch).
2. **Sample a huge number of candidate solutions** for a given problem — repeated sampling, but at a much larger scale than typically seen elsewhere.

3. **Filter and cluster** those candidates down to a small, manageable, and diverse set.
4. **Submit only that small set** for evaluation against the competition's hidden tests.

Step 1: Pre-Training and Fine-Tuning

AlphaCode's model was pre-trained on about **715 gigabytes of GitHub code**, using a masked language modeling objective (they also experimented with decoder-only models later, in AlphaCode 2), then fine-tuned on **CodeContests** — a large, curated dataset of competitive programming problems and their solutions.

A Fine-Tuning Trick: GOLD (Weighted Token Loss)

During fine-tuning, rather than treating every token equally in the next-token-prediction loss, AlphaCode used a technique called **GOLD**, which assigns **higher weight to higher-likelihood tokens** and lower weight to lower-likelihood ones. This regularization approach helps the model assign higher probability to more meaningful, common code patterns, improving precision in what it generates.

Step 2: Large-Scale Sampling

Once trained, AlphaCode generates an enormous number of candidate solutions per problem — **up to 1 million samples**, split roughly half in Python and half in C++. To maximize diversity across these samples:

- Problem tags and difficulty ratings in the prompt are **randomized**.
- A **high sampling temperature** is used, encouraging more varied outputs rather than many near-identical attempts.

Generating this many candidates is only practical because it's fully automated — no human effort scales with the sample count.

Step 3: Filtering and Clustering

Submitting 1 million solutions to an actual competition platform isn't possible — competitions typically allow only a handful of submissions per problem. So AlphaCode needed a way to narrow 1 million candidates down to a small handful of genuinely promising, diverse ones.

Two-Stage Narrowing

1. **Filter:** keep only the candidate solutions that pass the example tests already given in the problem statement — a first, cheap correctness check.

2. **Cluster:** among the surviving candidates, group solutions that are **syntactically different but semantically equivalent** — i.e., different code that does the same thing — so that the final submitted set represents genuinely different *approaches*, not just superficially different code doing the same thing repeatedly.

To support this clustering step, a **separate model** was trained specifically to generate plausible test inputs for a given problem description — helping distinguish solutions that behave differently under different inputs, which is a stronger signal of genuine semantic difference than just comparing source code text directly.

This clustering step matters a lot: if the final submission budget is small (e.g., 10 solutions), you want those 10 to be as diverse as possible, since submitting several near-duplicate solutions wastes attempts without meaningfully improving the chance of getting a correct one through.

Step 4: Evaluation

AlphaCode was evaluated in two ways:

- **CodeContests held-out test set** — since CodeContests was also used for training, a held-out portion gives a controlled internal signal of solution quality.
- **Live evaluation on Codeforces** — the real competition platform, where actual correctness and efficiency are benchmarked directly against human competitors.

Results

Submitting only **10 solutions per problem** (out of the full 1 million generated), across 10 real competitions with about 5,000 participants each, AlphaCode achieved an **average ranking of 54.3%** — meaning it performed better than roughly 45.7% of human competitors on average, and its performance was competitive with the **top 28%** of participants active in the last six months.

Results varied noticeably across different individual contests — in some, AlphaCode did very well; in others, considerably worse. Two plausible explanations for this variance: (1) how closely a given contest's problems resemble AlphaCode's training distribution, and (2) the filtering/clustering **selection stage itself can become a bottleneck** — some correct-ish (but not fully correct) solutions may get selected over genuinely correct ones, especially where the pool of near-miss solutions is large.

Measuring Search Quality: Pass@K vs. 10@K

Two related but distinct metrics come up here:

- **Pass@K:** if you generate K samples and could submit *all* of them, what fraction of problems would at least one of them solve? This measures the raw **coverage** of the sampling process itself — how good the model is at finding a correct solution somewhere in its outputs, ignoring the submission-limit constraint entirely.
- **10@K:** a more realistic metric — generate K total samples, but you're only allowed to actually **submit 10** of them (after filtering/clustering selects which 10). This measures not just whether a correct solution exists somewhere in the large sample pool, but whether the **selection process** is good enough to actually surface it within a small, realistic submission budget.

The gap between these two numbers reveals how much of a bottleneck the selection stage really is.

Scaling Results: Bigger Models and More Samples Both Help

Testing model sizes of 9B and 41B parameters (plus a 41B-with-clustering variant), measuring 10@K performance as K scales from 1,000 up to 1 million samples, on both a validation and test set:

- **Larger models consistently outperformed smaller ones**, holding sample count fixed — consistent with the general scaling trend covered earlier.
- **More samples consistently helped**, holding model size fixed — going from 10@1K to 10@1M meaningfully improved performance.
- **Clustering added a further, consistent improvement on top of both:** the 41B-with-clustering configuration outperformed the plain 41B model at every sample budget, since clustering specifically targets *diversity* within a limited 10-solution submission budget — which matters a lot for genuinely hard problems, where repeatedly submitting near-identical solutions wastes attempts.

The Selection Bottleneck, Quantified

Comparing 10@K (realistic, budget-limited) against pass@K (an idealized, unlimited-submission scenario) on the same sampling budget: pass@K reached somewhere above 40%, while 10@K topped out closer to 30% — a clear, measurable gap showing that the filtering-and-clustering selection stage leaves real performance on the table compared to what's theoretically available in the full sample pool.

The scaling pattern itself (solve rate vs. sample budget) follows the same log-linear trend seen in repeated-sampling research more broadly (see the Large Language Monkeys paper covered earlier) — and this log-linear relationship holds even *after* the selection/filtering stage, not just in the idealized full-coverage case.

Is Just Scaling Up Samples Enough?

A natural question: if 1 million samples works this well, would 1 trillion samples work even better? In principle, if the log-linear trend held indefinitely, yes — but there's a practical catch. **This assumes sampling more continues to produce more diverse solutions.** If sampling 10x more doesn't actually surface meaningfully different approaches (i.e., diversity saturates), then more samples stop translating into better coverage — the clustering step exists precisely to identify and select for this diversity, and its effectiveness is what actually determines whether scaling samples further keeps paying off.

In practice, **improving the underlying model** (as AlphaCode 2 did, using a more capable base model) tends to be a more efficient lever than just scaling the number of samples further — a stronger model needs far fewer samples to reach the same or better performance, since it's already generating higher-quality, more diverse candidates without brute-force sampling to compensate.

Summary: What AlphaCode Showed and Where It Struggled

Key Takeaways

- High coverage is achievable through **large-scale sampling combined with filtering and clustering** — this three-part combination was central to AlphaCode's results, not any single piece alone.
- The generated solutions showed genuine **novelty** — checks against the training data confirmed the model wasn't simply copying memorized solutions, suggesting real out-of-distribution generalization rather than retrieval.

Limitations

- **Training loss is a poor proxy for solve rate.** Many different solutions can solve the same problem, so optimizing next-token prediction loss doesn't directly optimize for what actually matters (getting a correct, efficient solution).
- **Uneven performance across problem types** — AlphaCode specifically struggled with domains like dynamic programming and constructive algorithms, showing that this approach isn't uniformly strong across all kinds of competitive programming problems.
- **The approach is resource-intensive** — generating and filtering from up to a million samples per problem isn't practical wherever there's a tight time or compute budget; a more realistic real-world deployment would need to rank and surface acceptable solutions incrementally as they're generated, rather than requiring the full large-scale sampling pass upfront.
- **Harder problems needed more than a single-shot generate-and-select approach.** For genuinely difficult problems, a **multi-step** solution process (rather

than one large parallel sampling pass) would likely perform better — foreshadowing the kinds of iterative, multi-step approaches covered in later research.

AlphaCode 2: A Different Strategy for the Same Problem

AlphaCode 2 (Google DeepMind, 2023) took a genuinely different approach to the same competitive-programming problem, and improved substantially on the original — worth examining closely because the differences reveal what actually moved the needle.

Key Change 1: Start From an Existing LLM Instead of Pre-Training From Scratch

Rather than pre-training a dedicated model from scratch (as the original AlphaCode did), AlphaCode 2 started from an existing large language model — **Gemini Pro** — and **fine-tuned** it for competitive programming, rather than relying purely on prompting.

Key Change 2: A Family of Models for Diversity

Instead of one fine-tuned model, AlphaCode 2 fine-tuned **several variants** with different hyperparameters (covering different difficulty levels and problem tags). Sampling from this whole family of models — rather than repeatedly sampling one model at high temperature — was used to boost diversity in the generated candidates.

Key Change 3: A Learned Scoring Model Instead of Just Filtering/Clustering

Rather than relying purely on the filter-and-cluster heuristic used in the original AlphaCode (pass the example tests, then cluster by behavioral similarity), AlphaCode 2 trained a **scoring model** — essentially a learned reward model — that estimates how likely a candidate solution is to be correct, on a scale from 0 to 1. This replaces a purely heuristic selection process with a **learned** one.

An Improved Dataset

Training used **CodeContests V2**, an improved, open-source version of the original dataset, along with a separate, more carefully human-curated, higher-quality dataset specifically used to train the scoring model.

The Updated Pipeline in Detail

1. **Data segmentation:** CodeContests V2 is split (tagged/segmented) into different subsets, used to fine-tune multiple different AlphaCode 2 model variants — this is what produces the "family of models" diversity mentioned above.
2. **Scoring model training:** the same underlying model and a separate, more heavily human-curated dataset are used to train the scoring model that estimates solution correctness.
3. **Sampling:** unlike the original AlphaCode's Python+C++ split, AlphaCode 2 samples **only in C++**. Temperature and prompt metadata are randomized across the family of models to maximize diversity in the generated candidates.
4. **Filtering:** each sample is actually executed against test inputs, and anything incorrect or that fails to compile is discarded — this step alone removes about **95%** of all generated samples.
5. **Clustering:** the surviving candidates (roughly 50 or so, after that heavy filtering) are aggregated into clusters, and the **top 10 largest clusters** are kept.
6. **Reranking:** within each retained cluster, the scoring model reranks candidates by estimated correctness, and the best candidate per cluster is selected as the final submission.
7. **Evaluation:** uses the same overall evaluation setup as the original AlphaCode (submission to Codeforces-style competitions).

Results: A Roughly 2x Improvement

Comparing solve rate against sampling budget: AlphaCode originally needed **1 million samples** to reach its best results. AlphaCode 2 reached the **same** solve rate using only about **100 samples** — and continued improving further as the sample budget increased beyond that.

At the same 1-million-sample budget, AlphaCode 2 reached a **43% solve rate**, compared to AlphaCode's **25%** — nearly double, using the same number of samples. The gains came from three compounding factors together: a stronger base model (Gemini Pro, fine-tuned), better diversity across a family of models, and a learned scoring model replacing purely heuristic selection.

Overall Standing vs. Human Competitors

AlphaCode 2 performed at approximately the **85th percentile** of human competitors overall — placing between the "Expert" and "Candidate Master" skill levels on Codeforces.

Open Questions and Discussion

Is Wasting 95% of Samples on Failed Compiles/Incorrect Answers a Problem?

This does look costly at first glance. Possible ways to reduce that waste, building on ideas from earlier notebooks:

- **Self-refinement instead of pure parallel sampling** — rather than generating a huge batch and discarding most of it, iteratively refine a smaller number of attempts using feedback (this reduces the number of samples needed, at the cost of needing a working feedback signal and more sequential time).
- **RL-based training** (as in train-time scaling, covered in an earlier notebook) — if the underlying model gets genuinely better at solving problems on its own, less test-time sampling is needed to reach the same level of performance in the first place.

Both approaches shift some of the burden from "brute-force test-time sampling" toward either better feedback loops or a stronger base model — with the trade-off that self-refinement requires figuring out what a useful feedback signal would even look like for a given problem.

Why Not Train the Scoring Model on the Exact Same Data as the Generator?

If the scoring model is trained on the *same* problems as the generator, there's a real risk of data contamination — the scoring model needs to generalize to genuinely new problems, not just recognize patterns from problems it already effectively "knows the answer to." It still needs data from a similar distribution to learn what a good vs. bad solution looks like, but training a separate, more curated dataset for scoring (as AlphaCode 2 did) helps avoid this overlap. Mixing datasets is possible in principle, but doing so across multiple fine-tuning stages introduces its own complications — most notably, a model can partially "forget" earlier-stage knowledge if later fine-tuning stages don't include some of the earlier data.

Practical Costs Remain

Even with major efficiency gains, this kind of experimentation is still expensive, and this whole approach is fairly specific to code — where automatic compilation and test execution give a cheap, reliable correctness signal that many other domains simply don't have.

Two Open Questions for Extending This Approach

Adapting Sampling Strategy to Problem Difficulty

One idea: use a separate classifier to label each problem as easy, medium, or hard, and adjust the **sampling strategy** accordingly — since easier problems likely need far fewer samples to reach good coverage (the model is more likely to land on a correct solution

quickly), while harder problems benefit from more extensive sampling. This connects to a pattern seen elsewhere in repeated-sampling research: coverage on easier problems saturates with relatively few samples, so scaling all problems up to the same huge sample budget wastes compute on the easy ones.

Embedding Reasoning Directly Into the Model

Rather than relying purely on massive external search (parallel sampling + selection), can the model be trained to reason its way toward a good solution more directly? A few possible directions:

- **Include algorithmic hints in training data** — showing the model, as part of training, what general approach/algorithm a given solution uses, not just the final code.
- **Chain-of-thought-style training data** — similar to the STaR-style approach covered earlier (generate a rationale, or backfill one using the answer as a hint), letting the model learn to "think before writing code," much like a human programmer plans an approach before implementing it.
- **Task decomposition for harder problems** — rather than trying to solve a hard problem in one shot, break it into subparts with hints for each part. This raises a real complication: if an early step is solved correctly but a later step isn't, how do you recover? This points toward needing some kind of **multi-step, tree-search-style approach** — sampling and checking at each step, with the ability to backtrack — rather than a single end-to-end generation. This is still an early, open direction rather than a solved problem, and task decomposition tends to work best when problems follow fairly common, previously-seen patterns; genuinely novel, out-of-distribution problem structures may still need a human in the loop.

Search-o1: Deep Research With Reasoning Models

Paper: arxiv.org/abs/2501.05366

Moving beyond code, this covers how to build a **deep research agent** — a system that combines a large reasoning model's step-by-step thinking with the ability to look things up when it doesn't know something.

Why Reasoning Models Alone Aren't Enough

Reasoning models have a **knowledge cutoff** — anything that happened after training simply isn't in the model's weights, and no amount of internal reasoning can recover it. A telling symptom: when a reasoning model hits a genuine knowledge gap, it tends to use **hedging language** — words like "perhaps," "possibly," "alternatively," or "wait" — right in its reasoning trace. On hard benchmarks like GPQA, these uncertainty markers show

up quite often (one analysis found over 30 such terms per reasoning trace, on average, for complex problems). Once a knowledge gap shows up this way, that uncertainty tends to **propagate through the rest of the reasoning chain**, undermining the final answer.

Why Plain Retrieval-Augmented Generation (RAG) Isn't Enough

The simplest fix would be standard **RAG**: take the question, generate a search query, fetch a relevant document, drop it into the prompt, and let the model answer using that document.

The Problem With Single-Shot Retrieval

This works reasonably well for simple factual questions (e.g., "what's the weather today?"), but breaks down for **complex, multi-step reasoning**. A hard problem might need different pieces of information at *different points* in the reasoning process — not just one lookup at the very start. Retrieving once, upfront, before any reasoning has happened, can't adapt to needs that only become clear partway through solving the problem. RAG tends to help over plain reasoning-without-retrieval, but that advantage shrinks or disappears for genuinely multi-step problems.

Search-o1's Two Key Ideas

1. Retrieve On the Go, Not Just Once

Instead of one retrieval step at the start, the model generates search queries **dynamically, as it reasons** — whenever it hits a genuine knowledge gap mid-reasoning, it triggers a search right there, gets a result, and continues. This can happen **multiple times within a single reasoning session**, rather than being a single fixed step.

2. Reason Over What Was Retrieved, Don't Just Dump It In

Rather than pasting a full retrieved document straight into the prompt, Search-o1 adds a separate **"Reason-in-Documents"** step: analyze the retrieved document, extract the specific relevant chunks of information, and only insert *that* distilled information back into the reasoning chain. This matters because dumping 10–20 full documents into a prompt can overwhelm the model — even if the right information is technically present somewhere in there, having too much irrelevant surrounding text can make it harder, not easier, for the model to reason well and land on the correct answer.

Worked Example: A Chemistry Problem

The question asks for the number of carbon atoms in a product resulting from a sequence of chemical reactions.

- **Vanilla reasoning (no retrieval):** the model hits an unfamiliar term, doesn't look anything up, and just guesses based on what's already in its weights — resulting in a wrong final answer, since the guess itself was wrong and that error cascades through the rest of the reasoning.
- **Simple retrieval (fetch and dump):** the model looks up the unfamiliar term and gets a document — but if too many loosely-related documents are fetched and all inserted into the prompt at once, there's too much content for the model to sift through effectively, and it can still land on a wrong answer.
- **Search-o1 (retrieve + reason-in-documents):** the model searches for the specific unfamiliar term, then extracts and refines just the relevant chunk of information (e.g., a specific chemical formula) before continuing its reasoning — integrating cleanly into the ongoing reasoning chain and reaching the correct answer.

The Two Components, Named

- **Agentic RAG:** the model inserts special marker tokens into its output whenever it hits enough uncertainty to trigger a search; a search query gets generated at that point, the retrieval tool returns documents, and those results get inserted back into the ongoing reasoning chain. This is the "search on the go, mid-reasoning" mechanism described above.
- **Reason-in-Documents:** a separate analysis step that processes each retrieved document, given the current query and what's been reasoned so far, and extracts only the genuinely relevant chunk before it gets appended back into the prompt — rather than the full raw document.

Together, these let the model behave a bit like a person doing research: not just collecting a pile of references, but actively taking notes on them and synthesizing the relevant parts before moving forward.

Results

Measured as pass@1 accuracy against number of documents retrieved, across physics, chemistry, and biology domains:

- For **plain reasoning** and **standard RAG**, accuracy generally does **not** improve as more documents are added — and can even get worse, since more raw content mostly adds noise rather than useful signal.
- For **Search-o1**, accuracy **does** improve as more documents are retrieved — because the reasoning step distills out what's actually relevant from each one, rather than being overwhelmed by volume.

Comparing to Human Experts on GPQA

Comparing Search-o1 (built on top of a reasoning model) against human expert performance on the GPQA benchmark, broken out by physics, chemistry, and biology: Search-o1 was competitive with — and in some cases exceeded — human expert-level scores in **physics** and **biology**, though it lagged further behind in **chemistry**, where meaningful headroom remained. (Framed carefully: this reflects being competitive with human experts on this specific class of hard benchmark questions, not a general claim of outperforming human expertise broadly.) Possible explanations discussed for the weaker chemistry performance: chemistry problems may be more sensitive to precise structural details that are easy to get subtly wrong, or the model's training data coverage for chemistry specifically may simply be weaker — this remains a testable, open question rather than a settled explanation.

Multi-Hop Question Answering

On benchmarks that require connecting information across multiple sources (HotpotQA, 2Wiki, MuSiQue, Bamboogle), both standard RAG and even the earlier Agentic RAG (without the Reason-in-Documents refinement step) tend to hit a performance ceiling. Search-o1 pushed past that ceiling, achieving the strongest reported results across several of these multi-hop benchmarks.

What's Actually Driving the Improvement?

Analyzing the reasoning chains directly: the frequency of uncertainty-signaling words ("perhaps," "wait," "likely," etc.) drops substantially with Search-o1 compared to plain reasoning — direct evidence that the approach is genuinely closing knowledge gaps, not just adding more text to the prompt. The gains come from two compounding factors: **better document quality** (via the Reason-in-Documents refinement) and the ability to **iteratively refine searches across multiple turns**, catching and correcting remaining uncertainty rather than settling for a single retrieval pass.

A Practical Caveat Worth Noting

This entire approach assumes the underlying retrieval system finds genuinely relevant documents in the first place — if retrieval itself returns loosely-related or unhelpful documents, no amount of downstream reasoning-over-documents can fully compensate. The paper's core claim is more specific: **even when multiple retrieved documents are only loosely relevant, simply dumping all of them into the context asks a lot of the model's reasoning** — and the Reason-in-Documents step is what helps manage that burden.

A Related Alternative: Search-R1

Search-o1 closes this loop through **prompting** — the model is instructed (via its scaffolding) on when and how to search and reason over documents, without any additional training. A related, unexplored-in-depth alternative is **Search-R1**, which instead teaches a model to search effectively using a **reinforcement learning**-based loop, rather than relying purely on prompting. The core difference: Search-o1 is a prompting-based system built on top of an existing reasoning model, while Search-R1 trains the search behavior directly into the model itself.

Open Questions

Do Models' Confidence Levels Actually Track Correctness?

If you look at a model's raw token-level probabilities (aggregated sensibly, not just summed) as a signal of confidence, models tend to be **overconfident** — if a model is actually correct only about 50% of the time on some class of questions, it might still report something like 80% confidence. This shows up practically too: when challenged on an answer, an overconfident model may stubbornly insist it's correct rather than reconsidering. There does appear to be **some correlation** between correctness and confidence (correct answers do tend to get relatively higher confidence than incorrect ones) — but the *miscalibration* (confidence running higher than actual accuracy) is a well-documented, ongoing pattern, and getting models to better calibrate their own confidence (including via additional RL/RLHF-style training specifically aimed at calibration) remains an active area of research.

Does RAG Change This Confidence Pattern?

This is flagged as a genuinely open, testable question — whether retrieval-augmented answers show different confidence/correctness correlation patterns than answers generated purely from the model's internal knowledge hasn't been definitively studied, and would make for a solid research project.